

Name: Y.Veerendar Reddy

Reg.No:- 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT - 16

Multi-Agent Reinforcement Learning Using Independent Q-Learning

Aim

To implement Independent Q-Learning for multiple agents and analyze how two agents learn to reach a common goal through interaction with the environment.

Procedure

1. Create a grid-based environment with two agents.
2. Define possible states and actions for each agent.
3. Initialize separate Q-tables for both agents.
4. Select actions using an ϵ -greedy strategy.
5. Move both agents according to their selected actions.
6. Calculate individual rewards.
7. Update each agent's Q-table using the Q-Learning equation.
8. Repeat the learning process for multiple episodes.
9. Compare the rewards and learned policies of both agents.
10. Visualize the learning performance.

Code:

```
# =====  
# EXPERIMENT 16  
# MULTI-AGENT REINFORCEMENT LEARNING  
# INDEPENDENT Q-LEARNING  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
np.random.seed(42)
```

```
# -----  
# Environment  
# -----  
  
grid_size = 4  
goal = (3, 3)  
  
n_states = grid_size * grid_size  
n_actions = 4  
  
# Actions: Up, Down, Left, Right  
moves = [  
    (-1, 0),  
    (1, 0),  
    (0, -1),  
    (0, 1)  
]  
  
# -----  
# Q-Tables for two independent agents  
# -----  
  
Q1 = np.zeros((n_states, n_actions))  
Q2 = np.zeros((n_states, n_actions))  
  
alpha = 0.1  
gamma = 0.9  
epsilon = 0.2  
  
episodes = 300  
max_steps = 30  
  
rewards1 = []  
rewards2 = []
```

```

# -----
# Convert position to state number
# -----

def state_number(position):
    return position[0] * grid_size + position[1]


# -----
# Move Agent
# -----

def move(position, action):

    r, c = position

    dr, dc = moves[action]

    r = np.clip(
        r + dr,
        0,
        grid_size - 1
    )

    c = np.clip(
        c + dc,
        0,
        grid_size - 1
    )

    return (r, c)

# -----

```

```

# Epsilon-Greedy Action
# -----

def choose_action(Q, state):

    if np.random.random() < epsilon:
        return np.random.randint(n_actions)

    return np.argmax(Q[state])

# =====
# TRAINING
# =====

for episode in range(epochs):

    agent1 = (0, 0)
    agent2 = (0, 1)

    total1 = 0
    total2 = 0

    for step in range(max_steps):

        s1 = state_number(agent1)
        s2 = state_number(agent2)

        # Select actions
        a1 = choose_action(Q1, s1)
        a2 = choose_action(Q2, s2)

        # Move agents
        next_agent1 = move(agent1, a1)
        next_agent2 = move(agent2, a2)

```

```

# Rewards

r1 = 10 if next_agent1 == goal else -1
r2 = 10 if next_agent2 == goal else -1


# Next states

ns1 = state_number(next_agent1)
ns2 = state_number(next_agent2)


# Q-Learning update

Q1[s1, a1] += alpha * (
    r1 +
    gamma * np.max(Q1[ns1]) -
    Q1[s1, a1]
)

Q2[s2, a2] += alpha * (
    r2 +
    gamma * np.max(Q2[ns2]) -
    Q2[s2, a2]
)

agent1 = next_agent1
agent2 = next_agent2

total1 += r1
total2 += r2

if agent1 == goal and agent2 == goal:
    break

rewards1.append(total1)
rewards2.append(total2)

# =====

```

```

# RESULTS
# =====

print("=" * 55)
print("INDEPENDENT Q-LEARNING RESULTS")
print("=" * 55)

print(
    "Agent 1 Average Reward:",
    round(np.mean(rewards1), 2)
)

print(
    "Agent 2 Average Reward:",
    round(np.mean(rewards2), 2)
)

print("\nAgent 1 Q-Table:")
print(Q1)

print("\nAgent 2 Q-Table:")
print(Q2)

# =====
# VISUALIZATION 1
# =====

plt.figure(figsize=(10, 5))

plt.plot(
    rewards1,
    label="Agent 1"
)

plt.plot(

```

```

    rewards2,
    label="Agent 2"
)

plt.title(
    "Independent Q-Learning Performance"
)

plt.xlabel("Episode")
plt.ylabel("Total Reward")

plt.legend()
plt.grid(True)

plt.show()

# =====
# VISUALIZATION 2
# =====

avg_window = 20

avg1 = np.convolve(
    rewards1,
    np.ones(avg_window) / avg_window,
    mode="valid"
)

avg2 = np.convolve(
    rewards2,
    np.ones(avg_window) / avg_window,
    mode="valid"
)

plt.figure(figsize=(10, 5))

```

```
plt.plot(  
    avg1,  
    label="Agent 1"  
)
```

```
plt.plot(  
    avg2,  
    label="Agent 2"  
)
```

```
plt.title(  
    "Moving Average Reward of Agents"  
)
```

```
plt.xlabel("Episode")  
plt.ylabel("Average Reward")
```

```
plt.legend()  
plt.grid(True)
```

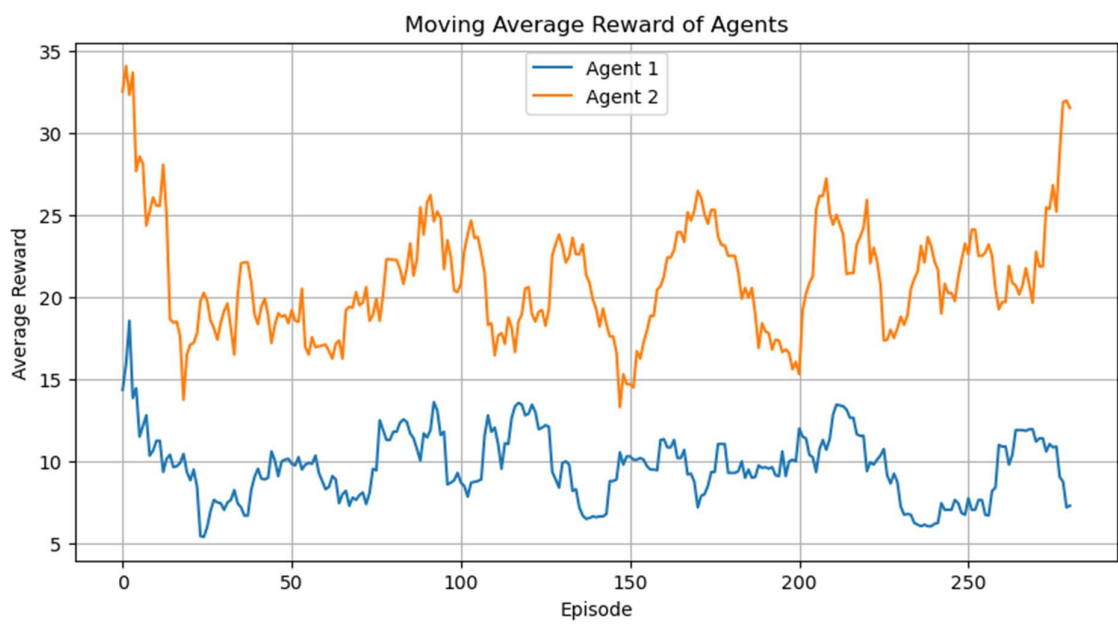
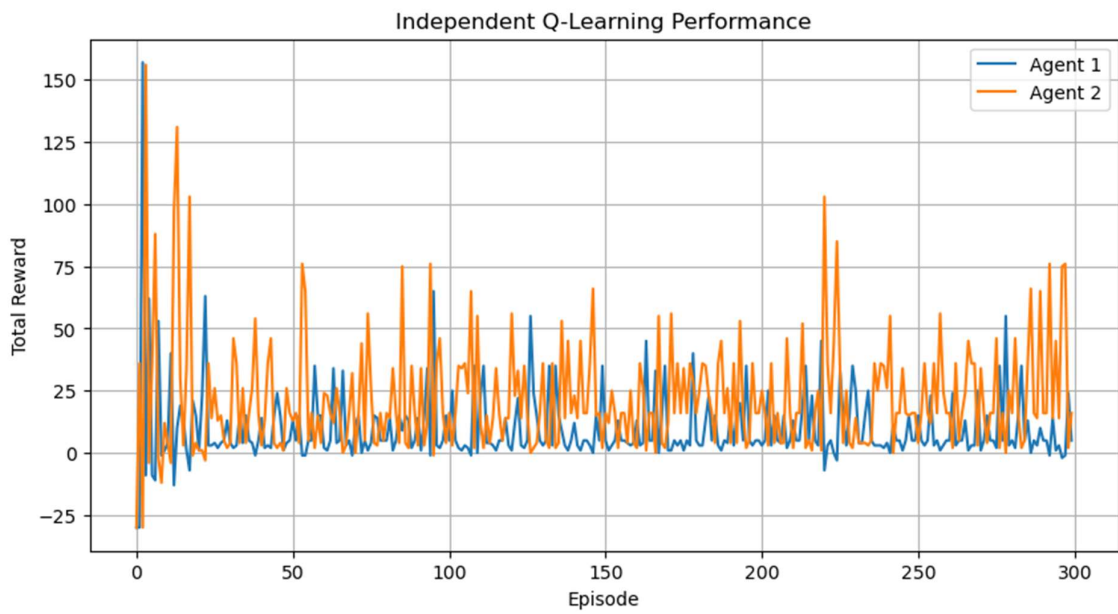
```
plt.show()
```


OUTPUT:-

```
=====
INDEPENDENT Q-LEARNING RESULTS
=====
Agent 1 Average Reward: 9.83
Agent 2 Average Reward: 21.75

Agent 1 Q-Table:
[[ 2.24932850e+01  4.30069472e+01  1.82555496e+01  1.96031826e+01]
 [ 1.94654224e+00  4.29282162e+01  8.19037228e+00  4.67623014e+00]
 [ 3.74668020e+00  3.82028706e+01 -6.88517838e-01  2.34261786e-01]
 [-6.47812836e-02  1.25841163e+01 -3.13120900e-01 -2.97010000e-01]
 [ 2.18530628e+01  5.81092151e+00  2.70400732e+01  4.93885864e+01]
 [ 2.36876943e+01  2.83839970e+01  2.91495877e+01  5.66927211e+01]
 [ 1.61513778e+01  6.48727754e+01  3.62249019e+01  3.29115734e+01]
 [ 5.81900220e-01  6.71289258e+01  7.18731188e+00 -1.99000000e-01]
 [-5.31369155e-01 -4.96009535e-01 -5.80446938e-01  2.52653631e+01]
 [ 2.69434963e+00  4.49991077e+00 -3.13120900e-01  5.84835228e+01]
 [ 3.10750109e+01  4.78989213e+01  2.70688125e+01  7.41345134e+01]
 [ 3.84133479e+01  8.44456728e+01  4.11785687e+01  4.49208411e+01]
 [-2.07100000e-01 -1.99000000e-01 -1.99000000e-01  1.18965647e-02]
 [-1.00000000e-01 -8.88249991e-02  0.00000000e+00  2.90396321e+01]
 [ 9.96019907e+00  6.96334595e+00  3.38267618e+00  7.67411161e+01]
 [ 3.10611966e+01  8.32866065e+01  3.39028639e+01  2.92066332e+01]]

Agent 2 Q-Table:
[[ 2.39060489e+00  2.20011706e+00  3.61827825e-01  4.17967053e+01]
 [ 3.44279507e+01  6.06736093e+01  2.05449467e+01  2.64415251e+01]
 [-5.85198506e-01  5.55242193e+01  1.23876274e+01 -5.78151100e-01]
 [-2.97010000e-01  1.23314206e+00 -3.21139900e-01 -2.97010000e-01]
 [ 4.45726072e+00 -5.99327029e-01 -5.85198506e-01  3.89333180e+01]
 [ 3.40788199e+01  1.90907193e+01  1.72141035e+01  6.87627880e+01]
 [ 1.82675834e+01  7.77435145e+01  4.54321808e+01  3.60474310e+01]
 [-2.07100000e-01  7.90252371e+01  1.37496548e+01  3.10001615e+00]
 [-3.29734810e-01 -3.69910000e-01 -3.94039900e-01 -3.57623096e-01]
 [ 2.75874108e+00 -3.69100000e-01 -1.99000000e-01  6.45329231e+01]
 [ 4.62959757e+01  5.17293442e+01  2.94126863e+01  8.77511834e+01]
 [ 4.27722797e+01  9.88937050e+01  4.99461739e+01  8.05899254e+01]
 [-3.05758000e-01 -1.99000000e-01 -1.99000000e-01 -1.99000000e-01]
 [-1.99000000e-01 -1.99000000e-01 -1.99000000e-01  8.88666464e+00]
 [ 7.20909464e+00  1.48524586e+01 -2.27441593e-02  9.51851841e+01]
 [ 6.68407172e+01  9.90469952e+01  6.93546368e+01  8.26038058e+01]]
```



Visualization

The program generates two graphs:

1. Independent Q-Learning Performance
Displays the total reward obtained by Agent 1 and Agent 2 in each episode.
2. Moving Average Reward
Shows the smoothed learning performance of both agents over multiple episodes.

Summary

Independent Q-Learning was implemented for two agents using separate Q-tables. Each agent independently learned the best action based on its own state, reward, and future expected value.

Result

Thus, Multi-Agent Reinforcement Learning using Independent Q-Learning was successfully implemented, and the learning performance of both agents was evaluated using reward-based visualizations.